Operating Systems - Lab Assignment 7

Threads, semaphores, deadlock, and priority scheduling

Name: Raj Pratap Singh Gurjar
Roll No.: 2401MC23

Overview

For this lab I added the four requested features to xv6-public and wrote a small user program for each one. I applied the patches to commit `eeb7b415dbcb12cc362d0783e41c3d1f44066b17`. Q1 to Q3 were run with two CPUs; I used one CPU for Q4 to make the scheduling order easier to see.

Q1 - Shared-address-space threads

I added `clone` and `join`. A cloned thread uses the same address space as the process, but has a separate kernel stack and the user stack passed to `clone`. The first stack frame returns through a thread-exit routine, and the parent joins each thread before the group can release its shared address space.

The test starts three threads, each doing 100,000 unsynchronized increments. My captured run finished at 203,864 instead of 300,000. The count changes from run to run because updates get lost when two threads access the counter at the same time.

Q2 - Semaphores

I added `sem_init`, `sem_wait`, and `sem_signal` using a fixed table of ten semaphores. A waiter sleeps while the count is zero and is woken by a signal. The protected counter finished at exactly 300,000. I also used the semaphores for a producer and consumer sharing a buffer of five slots; all 12 items were consumed.

Q3 - Deadlock detection and Banker safety

The program checks allocation and request matrices. In the first example it reports P0, P1, and P2 as deadlocked. In the second, it finds the safe order P2 -> P0 -> P1. The Banker example starts in a safe state, but the hypothetical request is denied because it would leave no safe completion order.

I also used two child processes and two pipes to make a circular wait. Each process keeps one pipe open and waits to read from the other. The program reports the cycle and cleans up the blocked children so the xv6 shell can be used again.

Q4 - Priority scheduling and aging

Priority 1 is highest and priority 20 is lowest. The scheduler chooses the highest-priority runnable process and rotates through the process table for ties. A process that has been runnable for more than 30 ticks gets one priority level better. In my single-CPU run, jobs starting at 1, 5, 10, and 20 finished in that order. The last job improved from priority 20 to 14 while it waited.

Build and run

I applied the four diffs in order to the pinned xv6-public commit, then built the kernel and all four user programs with the i686 ELF cross compiler and QEMU. The command in README.md includes three warning overrides for untouched xv6 files; without them, this newer compiler stops on warnings in the original code. The matching console output and screenshots are in this folder.

Results from my QEMU runs

Question	Observed result
Q1	203,864 / 300,000; the unsynchronized result varies.
Q2	300,000 protected increments; 12 of 12 items consumed.
Q3	Deadlock found; safe order printed; unsafe request denied.
Q4	Completion order 1, 5, 10, 20; aging was visible.

Note: Q1 is a race test, so the exact final count is not fixed. output_log.txt records the run shown here.